TEAM_POLARIS

1/3/26

What We Found

- 5-15% of all global trade is done via illicit supply chains, accounting for 3-5 Trillion USD annually.[1][2]
- The market for counterfeit goods is estimated to be as big as 497 billion USD. [1]
- Counterfeit or unethically-sourced goods like rare-earth metals, out-of-spec electronic components negatively impact company bottom lines in the era of increasing scrutiny and focus on sustainable supply chains. [3]
- The opactity of physical paper trails, complex cross-border transcontinental shipment routes & customs processes make end-to-end transperency and lifecycle management of goods impossible.

What We Forsaw

- A devised a solution in the form of a proof-of-work, consortium blockchain which could:

  > Track a good from raw materials to recycling.

  > Reducing supply-chain tampering.

  > Ensuring that raw materials are sourced sustainably by tracking them every step of the way.

  > Helping companies protect their bottom line from forces like opaque tarrifs, theft, etc.

- In essence, a rare win-win situation for both customer and producers.

What We Did

In the past 24 hours, our team has managed to build not just a platform but an entire ecosystem involving:

> A extremely memory-safe and memory-efficient backend.

> A database storage system with sub-millisecond latency and permanence.

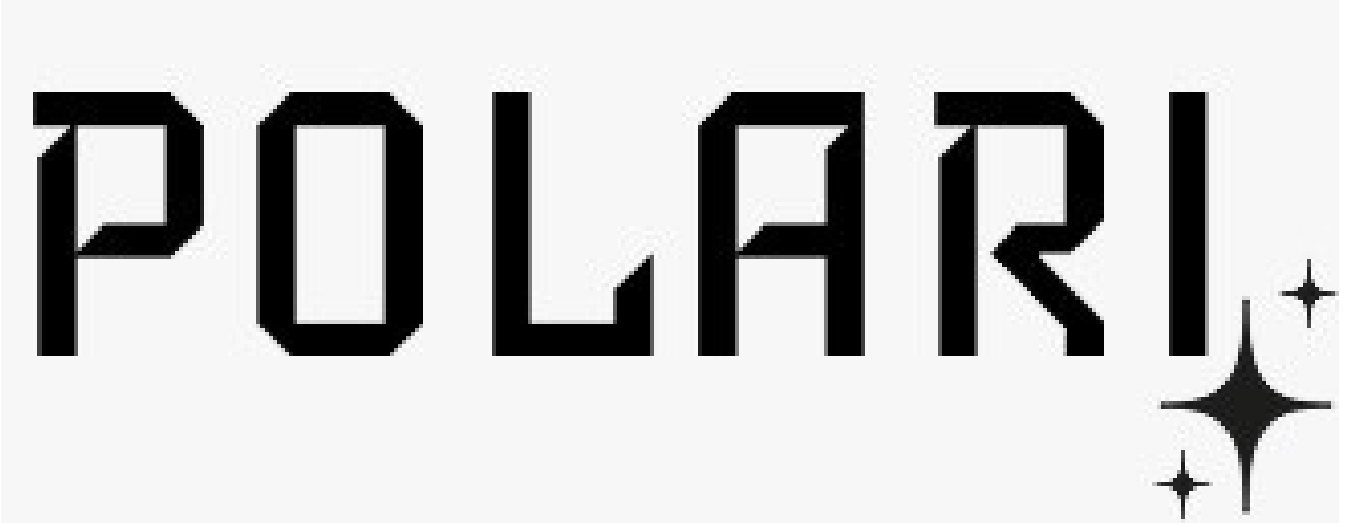
> Implemented a decentralized peer-to-peer syncing solution ensuring that multiple ledgers are maintained independently but in sync.

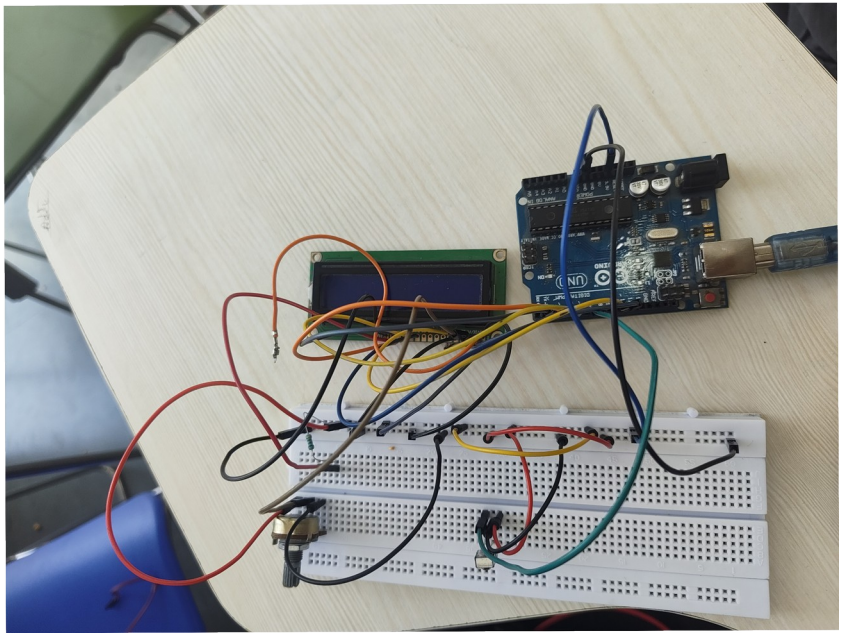
> Implemented multiple client accessable avenues including

>> desktop client for brand storefronts
>> mobile web-app with QR-code inputs for delivery

>> an embedded computing solution for warehouses
>> dev-friendly RESTful APIs
>> & even shell scripts for a CLI interface and extending our ecosystem as needs arise in the future.

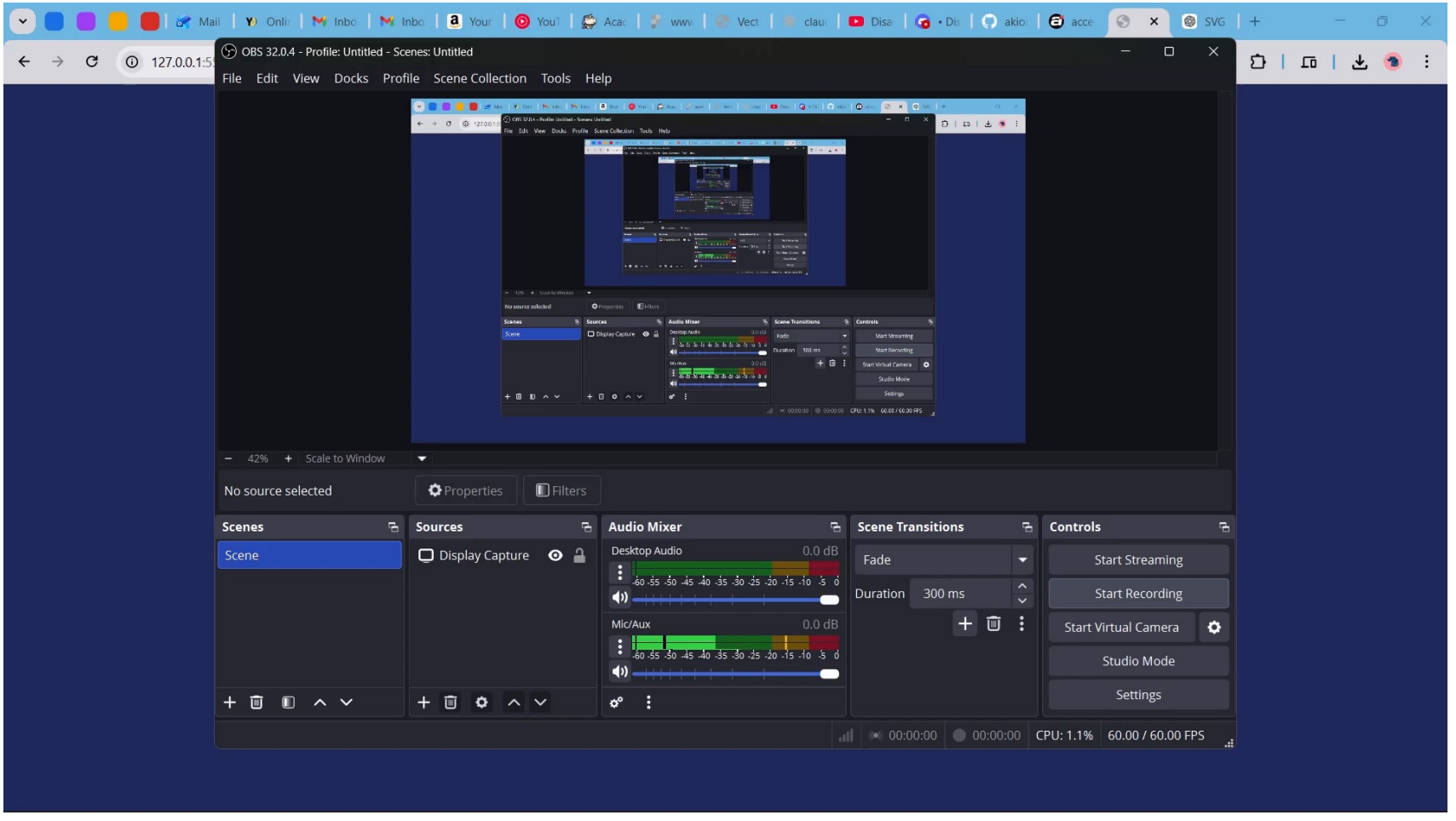
We Call It





```
File Edit Selection View Go Run Terminal Help
polari
EXPLORER
POLARI
python_server
__pycache__
bin
include / python3.13
lib
lib64
.gitignore
get_data_redis.py
main.py
py_client_embedde...
pyvenv.cfg
test.py
try.sh
src
block.rs U
blockchain.rs 2, U
lib.rs U
main.rs U
net_message.rs U
p2p.rs 2, U
target
.gitignore U
Cargo.lock U
Cargo.toml U
OUTLINE
TIMELINE
Cargo.toml U | lib.rs U | block.rs U | blockchain.rs 2, U | net_message.rs U | ma
src > block.rs > ...
28  impl Block {
29      pub fn new(index: u64, data: SupplyChainData, previous_hash: String) -> Self {
31          let mut block: Block = Block {
32              index,
33              timestamp,
34              proof_of_work: 0,
35              previous_hash,
36              data,
37              hash: String::new(),
38          };
39          block.hash = block.calculate_hash();
40          block
41      }
42
43      pub fn calculate_hash(&self) -> String {
44          let input: String = serde_json::to_string(&BlockForHashing {
45              index: self.index,
46              timestamp: self.timestamp.clone(),
47              proof_of_work: self.proof_of_work,
48              previous_hash: self.previous_hash.clone(),
49              data: self.data.clone(),
50          }).Result<String, Error>
51           .unwrap();
52
53          let mut hasher: CoreWrapper<CtVariableCoreWrapper<…, …, …> = Sha256::new();
54          hasher.update(data: input.as_bytes());
55          hex::encode(data: hasher.finalize())
56      }
57
58      pub fn mine_block(&mut self, difficulty: usize) {
59          let target: String = "0".repeat(difficulty);
60          while !self.hash.starts_with(&target) {
61              self.proof_of_work += 1;
62              self.hash = self.calculate_hash();
63          }
64      }
65
66      pub fn is_valid_block(&self, previous_block: &Block, difficulty: usize) -> bool {
TERMINAL
(python_server) ➜ polari git:(master) ✗ cargo run
  |
15 | impl Blockchain {
  | --------------- method in this implementation
...
135 |     pub fn get_item_trace(&self, item_id: &str) ...
  |            ^^^^^^^^^^^^^^
  |
  = note: `#[warn(dead_code)]` (part of `#[warn(unused)]`) on by default

warning: method `broadcast_block` is never used
  --> src/p2p.rs:90:12
  |
34 | impl P2PNode {
  | ------------ method in this implementation
...
90 |     pub fn broadcast_block(&mut self, block: Bloc...
  |            ^^^^^^^^^^^^^^

warning: `polari` (bin "polari") generated 4 warnings (1 duplicate)
    Finished `dev` profile [unoptimized + debuginfo] target(s) in 0.33s
warning: the following packages contain code that will be rejected by a future version of Rust: redis v0.23.3
note: to see what the problems were, use the option `--future-incompat-report`, or run `cargo report future-incompatibilities --id 1`
     Running `target/debug/polari`
zsh pyt...
target/deb...
Ln 91, Col 1  Spaces: 4  UTF-8  LF  { } Rust  Prettier  Dependi
```

```
File Edit Selection View Go Run Terminal Help
python_server
EXPLORER
PYTHON_SERVER
__pycache__
bin
include
lib
lib64
.gitignore
get_data_redis.py
main.py
py_client_embedded....
pyvenv.cfg
test.py
try.sh
OUTLINE
TIMELINE
main.py | try.sh | get_data_redis.py | test.py | py_client_embedded.py
$ try.sh
1  #!/bin/bash
2
3  # Colors for output
4  RED='\033[0;31m'
5  GREEN='\033[0;32m'
6  YELLOW='\033[1;33m'
7  NC='\033[0m' # No Color
8
9  echo -e "${YELLOW}Supply Chain Event Adder${NC}"
10 echo "========================"
11 echo
12
13 # Get item details from user
14 read -p "Item ID (e.g. ECID123): " ITEM_ID
15 read -p "Event Type (manufacture/ship/receive): " EVENT_TYPE
16 read -p "Location (e.g. Factory Delhi): " LOCATION
17 read -p "Owner (e.g. SupplierA): " OWNER
18 read -p "Document Hash (or press Enter for auto): " DOC_HASH
19
20 # Auto-generate timestamp and doc hash if empty
21 TIMESTAMP=$(date -Iseconds -u)
22 DOC_HASH=${DOC_HASH:-"auto_$(date +%s)_$RANDOM"}
23
24 # JSON payload for your Rust app
25 EVENT_JSON=$(cat <<EOF
26 {
27   "item_id": "$ITEM_ID",
28   "event_type": "$EVENT_TYPE",
29   "location": "$LOCATION",
30   "timestamp": "$TIMESTAMP",
31   "owner": "$OWNER",
32   "document_hash": "$DOC_HASH"
33 }
34 EOF
35 )
36
37 echo -e "\n${GREEN}Adding event:${NC}"
38 echo "$EVENT_JSON"
TERMINAL
(python_server) ➜ python_server git:(master) ✗ uvicorn main:app --host 0.0.0.0 --port 8000 --reload
  |
  = note: `#[warn(dead_code)]` (part of `#[warn(unused)]`) on by default

warning: method `broadcast_block` is never used
  --> src/p2p.rs:90:12
  |
34 | impl P2PNode {
  | ------------ method in this implementation
...
90 |     pub fn broadcast_block(&mut self, block: Bloc...
  |            ^^^^^^^^^^^^^^

warning: `polari` (bin "polari") generated 4 warnings (1 duplicate)
    Finished `dev` profile [unoptimized + debuginfo] target(s) in 0.36s
warning: the following packages contain code that will be rejected by a future version of Rust: redis v0.23.3
note: to see what the problems were, use the option `--future-incompat-report`, or run `cargo report future-incompatibilities --id 1`
     Running `/home/akionsight/Desktop/code/polari/target/debug/polari add a a a a a auto_1772337438_8086`
Added new block. Chain length: 14
Chain valid: true
INFO:     10.75.55.232:57477 - "POST /add-item/ HTTP/1.1" 200 OK
INFO:     10.75.55.232:62812 - "GET /items/?itemid=ECID123 HTTP/1.1" 200 OK
INFO:     10.75.55.232:50253 - "GET /items/?itemid=ECID123 HTTP/1.1" 200 OK
INFO:     10.75.55.232:65454 - "GET /items/?itemid=ECID123 HTTP/1.1" 200 OK
zsh
Python: ge...
zsh
Ln 2, Col 1  Spaces: 4  UTF-8  LF  { } Shell Script  Prettier
```


OBS 32.0.4 - Profile: Untitled - Scenes: Untitled
File Edit View Docks Profile Scene Collection Tools Help
42% Scale to Window
No source selected
Properties
Filters
Scenes
Scene
Sources
Display Capture
Audio Mixer
Desktop Audio 0.0 dB
Mic/Aux 0.0 dB
Scene Transitions
Fade
Duration 300 ms
Controls
Start Streaming
Start Recording
Start Virtual Camera
Studio Mode
Settings
00:00:00
00:00:00
CPU: 1.1%
60.00 / 60.00 FPS

What's Next

- We plan to implement smart contracts on our blockchain so as to allow for automatic contract execution (eg: for allowing automatic payment dispatch as soon as goods reach the retailer)
- We also plan to make our p2p networks more robust and allow for node discovery across the internet
- Backing up ledgers to an ACID compliant database.
- Make nodes easily deployable via containerized solutions like docker

Sources

- [1] OECD. (n.d.). Counterfeit and pirated goods. OECD. https://www.oecd.org/en/topics/counterfeit-and-pirated-goods.html
- [2] EXPOSING SUPPLY CHAIN VULNERABILITIES TO ILLICIT TRADE A GLOBAL REPORT ON DYNAMICS, HOTSPOTS AND RESPONSES ACROSS 10 SECTORS EXECUTIVE SUMMARY. (n.d.). Retrieved May 25, 2025, from https://www.tracit.org/uploads/1/0/2/2/102238034/tracit_exposing_supply_chain_vulnerabilities_to_illicit_trade_nov2023_executive_summary.pdf
- [3] Ghadge, A., Duck, A., Er, M., & Caldwell, N. (2021). Deceptive counterfeit risk in global supply chains. Supply Chain Forum: An International Journal, 22(2), 1–13. https://doi.org/10.1080/16258312.2021.1908844
- [4] Lee, Henry, Global Sea Freight Business Digital Transformation for Electronic Manufacturing Services (EMS) Industry (March 19, 2025). Available at SSRN: https://ssrn.com/abstract=5241413 or http://dx.doi.org/10.2139/ssrn.5241413

Thank You